GRANT TRUST SYSTEM

A Control Protocol for Autonomous Agent Output in Research Ecosystems

Caleb Strom · Shon Pan · 2026

Apart Research Hackathon · **Track 2: Control Protocol Design** · Track 3: Building Better Red Teams

Core contribution: Autonomous research agents write grant applications, literature reviews, and experiment protocols on behalf of human principals. Without verification infrastructure, funders and institutions cannot distinguish genuine human judgment from careless automation. The Grant Trust System is a control protocol that makes this distinction detectable and actionable — without banning AI use or relying on a detection arms race.

1 The Control Problem

Automated Research Institutes (ARIs) pair human researchers with AI agent suites that run experiments, collect data, and draft outputs. This creates a specific control problem at the human–funder boundary: the grant writing agent is a high-autonomy system whose outputs determine real resource allocation, with limited principal oversight of the path taken.

The threat is a spectrum of misalignment between agent output and the researcher's genuine scientific judgment — from minor stylistic drift to fully automated applications the researcher approves without meaningful review. This maps directly to ControlArena-style settings: the agent's main task is a complex language task (produce a quality application); the side task is volume maximization, optimizing for a proxy (submission count) the principal cannot cheaply observe.

Threat model: A grant writing agent maximizing application volume over fit is exhibiting misaligned side-task behavior. Neither the human principal (operator) nor the funder (second principal) can detect this from the output text alone. The goal of this protocol is to make that detection tractable.

A naive control response is to build an AI detector. This fails: any capable agent optimizes against a known detector, and the adversary has lower costs. The Grant Trust System avoids the arms race by measuring the costliness of behavioral signals associated with genuine principal oversight — signals that cannot be cheaply faked even by a capable agent because they require real-world human action to produce.

2 System Architecture

The protocol has three layers composing into a single control gate:

```
  Match Quality = 0.35 × Intent Score  +  0.40 × Fit Score  +  0.25 × Reputation
Score

  ≥ 0.65  →  Pass control gate (surface to funder)
  < 0.65  →  Reject with improvement pathway (feedback loop to re-apply)
```

Layer	Control function	What it detects	Owner
1. Intent Detection	Monitor for agent-vs-human signal	Template use, voice divergence, spray patterns	Caleb Strom
2. Fit + Reputation	Costly signal verification	Funder alignment, verified track record	Shon Pan
3. Improvement Pathways	Feedback and development loop	Unblocks principals; updates ledger on completion	Shon Pan

Control Flow

```
  Agent-generated application
    ├─ [Layer 1: Intent Detection] ──────────────────────────────────────────┐
    │      DBSCAN vs LLM baselines → Template score (known attack pattern?)
│
```

```
    │     Cosine vs researcher corpus → Authorship score (principal's voice?)
│
    │     Cross-application uniformity → Spray score (volume side task?)
│
    │
│
    ├─ [Layer 2: Fit + Reputation] ──────────────────────────────────────-
    │     Funder mission match → Fit score
│
    │     Hackathons, repos, publications → Reputation score (costly signals)
│
    │
│
    └─ [Control gate: Match Quality]
│
          PASS → surface to funder
│
          FAIL → improvement pathway → update ledger → re-apply ──────────────┘
```

3 Layer 1: The Intent Detection Engine

3.1 Monitor Design: The Xenarch Approach

The Intent Detection Engine (IDE) adapts the Xenarch anomaly detection pipeline (Strom, 2026) from planetary imagery to text. Xenarch trains a VAE exclusively on natural geology and detects deviations without ever training on artificial examples. The IDE trains on natural human writing (the researcher's own corpus) and detects applications that deviate from that personal distribution, or that cluster with known LLM output patterns. The monitor never needs to classify a specific model directly, avoiding the detection arms race.

Xenarch concept	IDE analogy	Control interpretation
Natural geology corpus	Researcher prior writing	Ground truth for the principal's voice
DBSCAN vs. baselines	Template detection	Did the agent use a known attack pattern?
Reconstruction error	Authorship divergence	Output distance from the principal's distribution
Contextual metric (30%)	Specificity score	Genuine engagement or generic proxy behavior?
Cross-chip uniformity	Spray detection	Agent optimizing volume (side task) over quality?

3.2 Feature Extraction: 42 Stylometric Features

Four families, each targeting a distinct agent failure mode:

- Lexical richness (10): type-token ratio, hapax rate, Yule's K, contraction rate, frequency entropy. Agents converge on intermediate vocabulary diversity — lower idiosyncrasy than authentic human writing.
- Syntactic patterns (12): sentence length mean, std, entropy; punctuation rates; paragraph structure. Unnaturally low sentence length variance is the primary spray-and-pray indicator.
- Surface statistics (8): Flesch-Kincaid approximation, digit density, log-normalized length, uppercase ratio.
- Semantic texture (12): LLM cliché density; named entity density; numeric specificity; temporal density; evidence rate (URLs, citations); funder-specific language. The highest-signal family — analogous to Xenarch's contextual metric.

3.3 The Four Monitor Signals

Signal	Formula	What it catches	Control flag
Authorship	Cosine(app, corpus_mean) − semantic_penalty	Agent output without principal review	LOW_AUTHORSHIP
Template distance	tanh(dist_to_LLM_cluster) − cliché_density	Known agent behavioral pattern used	TEMPLATE_MATCH
Spray (inverted)	Cross-app pairwise similarity + opener score	Volume maximization side task	COPY_PASTE_RISK
Specificity	Numbers + citations + NEs + funder keywords	Generic proxy behavior, no real engagement	LOW_SPECIFICITY

```
Intent Score = 0.30 × authorship  +  0.25 × template_distance
             + 0.25 × (1 − spray)  +  0.20 × specificity

[0.00, 0.25)  →  spray_and_pray     — agent side task detected, high confidence
[0.25, 0.45)  →  likely_automated   — low principal oversight
[0.45, 0.60)  →  mixed              — ambiguous; trigger interrogation protocol
[0.60, 0.80)  →  likely_genuine     — principal engagement evident
[0.80, 1.00]  →  genuine            — strong human-in-the-loop signal
```

4 Monitor Validation

The monitor was tested against three synthetic principals spanning the control-relevant range. A fourth case tests the feedback loop after principal-side intervention.

Principal	Auth.	Tmpl.	Spray	Spec.	Intent	Label	Gate
A — Genuine researcher, AI-assisted	0.958	0.978	0.275	0.209	0.755	likely_genuine	✓
B — Agent spray-and-pray, no corpus	0.500*	0.795	0.692	0.029	0.415	likely_automated	×
C — Genuine, mismatched funder	0.969	0.978	0.458	0.121	0.693	likely_genuine	†
B — Post-hackathon intervention	0.600	0.850	0.380	0.180	0.555	mixed	↺

** No corpus: authorship defaults to 0.50 (neutral). Spray and template signals correctly classify B independently. † Passes intent gate; redirected to better-matched funder. ↺ Feedback loop: intervention moves B from likely_automated to mixed.*

Key control protocol observations: (1) The monitor correctly identifies volume-maximization side-task behavior in case B without access to agent logs. (2) Case A confirms the protocol does not penalize AI use — only unsupervised AI use. (3) The post-intervention case demonstrates the feedback loop: principal-side actions change the monitor output, making the protocol developmental rather than purely restrictive.

Monitor gap — Track 2 extension: The mixed region (0.45–0.60) is ambiguous. This is the natural target for an interrogation protocol: when the monitor is uncertain, a trusted model asks the principal targeted factual questions about the application before the control gate fires. This directly implements the Track 2 objective of testing interrogation protocols where the trusted model questions the untrusted model before executing an action.

5 Layers 2 and 3: Costly Signals and Feedback

5.1 The Reputation Ledger as a Hard Control Floor

Stylometric monitoring can be evaded by a sufficiently capable agent. The reputation ledger is the control floor that remains robust under adversarial conditions. It tracks signals that are costly to produce at scale:

- Hackathon participations with verifiable shipped outputs (Devpost records)
- Open source contributions (GitHub commit history, project usage)
- Published work with independent engagement signals (citations, downloads)
- Verified collaborations (co-authorship, institutional records)

The core asymmetry: An agent can produce 200 high-quality applications in an afternoon. It cannot produce 200 shipped open source projects or three years of commit history. Even a perfect stylometric attack cannot overcome a missing track record. The ledger and the intent monitor are defense-in-depth: both must be defeated simultaneously.

Three red team attack vectors and their mitigations: (1) Stylometric evasion — craft output matching the principal's voice. Partially defeated by spray detection; fully defeated by the ledger. (2) Fake corpus injection — manufacture prior writing to anchor authorship scoring. Mitigated by corpus provenance verification against dated external records. (3) Ledger gaming — fabricate hackathon or GitHub activity. At scale, this costs more than the legitimate behavior it mimics.

5.2 Improvement Pathways: Measuring Safety / Usefulness Tradeoff

A control protocol that only blocks is incomplete. Below-threshold principals receive actionable next steps rather than flat rejections. The generator branches on the lowest-scoring component: low intent triggers writing guidance, low reputation triggers hackathon recommendations, low fit redirects to better-matched funders. Completed actions update the reputation ledger.

This is the Track 2 safety/usefulness tradeoff in concrete form. The improvement pathway system converts blocked applications into a development pipeline, reducing the false-positive cost of the control protocol over time.

6 Generalization: ARIs as a Multi-Agent Control Setting

The Grant Trust System is the trust infrastructure layer of a full Automated Research Institute. ARIs pair each human researcher with a personal agent suite (experiment, literature, data collection, grant writing agents). The trust infrastructure generalizes across every agent output type.

```
AUTOMATED RESEARCH INSTITUTE
├── HUMAN COLLABORATION  ──  Peer review · co-authorship · seminars
├── RESEARCHER PODS      ──  Experiment · Literature · Data · Grant agents
├── TRUST INFRASTRUCTURE ──  Grant Trust System (this work)
│     Intent Detection · Reputation Ledger · Improvement Pathways
└── GOVERNANCE           ──  Approval gates · audit trail · ethics
```

ARI output	Monitor question	Protocol generalization
Grant applications	Genuine principal engagement?	Intent Detection Engine (current)
Literature annotations	Did principal read and judge these papers?	Authorship scorer vs. annotation corpus
Experiment protocols	Did principal design this, or rubber-stamp agent draft?	Revision pattern analysis, specificity scoring
Peer review outputs	Genuine scholarly judgment?	Voice consistency vs. publication record
Publications	Genuine research contribution?	Full-document stylometric fingerprinting

ARIs also instantiate the multi-agent collusion control problem (Track 1): multiple agent pods could coordinate to produce stylometrically consistent but fabricated materials, defeating per-application monitoring. The reputation ledger's use of independently verifiable costly signals is a direct defense, as fabricated inter-agent collusion cannot cheaply produce real-world track records.

7 Limitations and Track-Mapped Future Work

Known Limitations

- Corpus dependency: authorship score defaults to neutral (0.50) with no prior corpus. Weight reputation signals more heavily for new principals; provide corpus-building guidance.
- Mixed-region ambiguity (0.45–0.60): target zone for interrogation protocol extension (see below).
- Adversarial robustness: the reputation ledger is the floor. A capable agent can shift stylometric scores but not fabricate a track record at scale.
- Linguistic bias: features calibrated on English academic writing may disadvantage non-native speakers. Mitigate with language-matched corpus calibration.

Proposed Extensions by Track

Track 2 — Interrogation Protocol for Mixed Scores

When the monitor returns 0.45–0.60, trigger a trusted-model interrogation: ask the principal specific factual questions about claims in the application (What is your model's RMSE? How many deployment sites? What is your training set size?). Answers are compared against the application text and the researcher's corpus. This directly tests the Track 2 objective of interrogation protocols where the trusted model questions the untrusted model before action.

Track 3 — Red Team Dataset Construction

Generate a labeled dataset of agent-produced applications attempting to evade each monitor signal independently, then in combination. Measure the minimum costly-signal footprint required to defeat the full protocol. This produces the Track 3 deliverable: a dataset of sneaky transcripts that a prompted monitor cannot catch, with analysis of which feature families are most vulnerable.

Track 2 — Monitor Generalization

Test whether the stylometric monitor trained on grant applications generalizes to other ARI output types. Identify which feature families transfer across domains and which require domain-specific calibration. This addresses the Track 2 objective of training monitors that generalize to other attack datasets.

What this proves for AI control: Monitoring agent outputs for human-in-the-loop signal is tractable with standard NLP tools — no access to agent internals required. The costly signal layer provides adversarial robustness that pure stylometric monitoring cannot guarantee. Together they form a practical, deployable control protocol for high-stakes agent-generated outputs in real research environments.

References

Greenblatt, R., et al. (2024). AI Control: Improving Safety Despite Intentional Subversion. arXiv:2312.06942.

Strom, C. (2026). Xenarch Model: Automated Detection of Planetary Surface Technosignatures with Stable Deep Learning. Preprint.

Burrows, J. (2002). Delta: A measure of stylistic difference and a guide to likely authorship. Literary and Linguistic Computing, 17(3), 267–287.

Argamon, S., & Juola, P. (2011). Overview of the international authorship identification competition at PAN-2011. CLEF 2011.

Ester, M., et al. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. KDD.

Kingma, D. P., & Welling, M. (2014). Auto-encoding variational Bayes. ICLR.